II. Heuristic Methods

We continue with our problem of optimization.

Throughout, Ω is a closed and bounded subset of $\mathbb{R}^n$, and $f:\Omega \to \mathbb{R}$ a continuous function.

We call Ω the *feasible domain*, and f the *objective function*, and write the problem as follows:

objective function

$$\text{Maximize / minimize} \quad f(x)$$

$$\text{Subject to} \quad x \in \Omega$$

feasible domain

The Extreme Value Theorem guarantees:

f takes a maximum and a minimum value on Ω.

In optimization, often one finds difficulties such as:

- the function f is not differentiable,
- there is no exact formula for the objective function. Instead one is given an algorithm that computes the value of the function at each point.
- One works in a high dimensional space, and the common algorithms require too much memory and computation time (complexity).

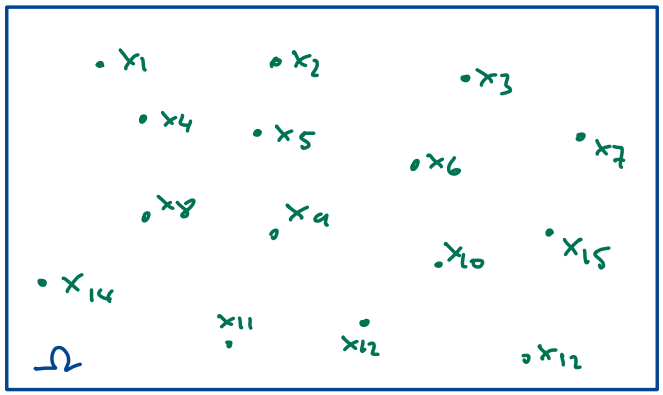
Heuristic algorimthms can be used in such situations. They are algorithms designed to find *approximate solutions* by "trial and error", that is, by *random methods*. The algorithms use strategies to improve performance which often are inspired by natural processes, or by animal behavior, for example.

II.1 Naïve Random Search

Naïve random search algorithms are totally random, without strategy.

a) Basic naïve random search algorithm:

1. Pick a collection of sample points $x_1, \ldots, x_N$ in Ω by random.
2. Evaluate the objective function at these points.
3. The smallest/largest of the values of $f(x_1), \ldots, f(x_N)$ will be the max/min.



Choose points $x_1, \ldots, x_{15}$ in Ω by random

Compare

$f(x_1), f(x_2), f(x_3), \ldots, f(x_{15})$

which is largest?

which is smallest?

Example: Find the min. value of $\boxed{f(x) = x^2}$ on $\Omega = [-5, 5]$

Sol.: We pick 15 numbers in $[-5, 5]$ by <u>random</u>

$x_1 = 1.12$ $\quad f(1.12) = 1.2544$

$x_2 = -4.78$ $\quad f(-4.78) = 22.8484$

$x_3 = 0.24$ $\quad f(0.24) = 0.0576$

$x_4 = 1.74$ $\quad f(1.74) = 3.0276$

$x_5 = -3.22$ $\quad f(-3.22) = 10.3684$

$x_6 = 1.87$ $\quad f(1.87) = 3.4969$

$x_7 = 0.78$ $\quad f(0.78) = 0.6084$

$x_8 = -2.13$ $\quad f(-2.13) = 4.5369$

$x_9 = 4.22$ $\quad f(4.22) = 17.8084$

$x_{10} = 3.72$ $\quad f(3.72) = 13.8384$

$x_{11} = -0.12$ $\quad f(-0.12) = 0.0144$

$x_{12} = -1.12$ $\quad f(-1.12) = 1.2544$

$x_{13} = 2.73$ $\quad f(2.73) = 7.4529$

$x_{14} = 3.04$ $\quad f(3.04) = 9.2416$

$x_{15} = -2.22$ $\quad f(-2.22) = 4.9284$

The <u>smallest</u> of these numbers is

$f(-0.12) = 0.0144$ ← estimate of the <u>absolute</u> <u>min</u> value of $f(x) = x^2$ on $[-5, 5]$

The largest of these numbers is

$f(-4.78) = 22.8484$ ← estimate of the absolute <u>max</u> value of $f(x) = x^2$ on $[-5, 5]$.

These estimates have large errors because the number of sample points x_k is <u>too</u> small.

Advantages (of the basic naïve random search):

- A very simple algorithm
- Does not require much memory
- Computations can be done in parallel

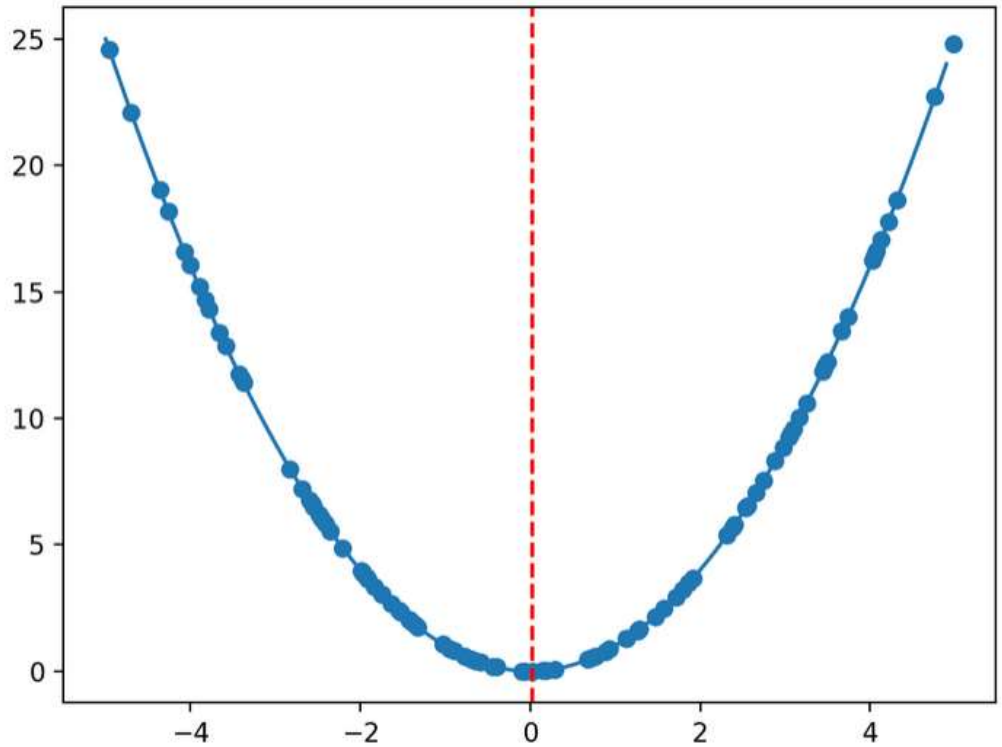
Disadvantage:

For good results, one needs an extremely large number N of sample points, which makes this algorithm extremely slow. This is particularly true in high dimensions.

```
# example of naïve random search for minimization
from numpy import arange
from numpy.random import rand
from matplotlib import pyplot

# objective function
def f(x):
    return x**2.0

# the feasible domain Omega is an interval [r_min,r_max]
r_min, r_max = -5.0, 5.0
# generate a random sample of 100 points from the domain
sample = r_min + rand(100) * (r_max - r_min)
# evaluate the sample
sample_eval = f(sample)
#
# locate the best solution within sample_eval
best_ix = 0
for i in range(len(sample)):
    if sample_eval[i] < sample_eval[best_ix]:
        best_ix = i
#
# summarize best solution
print('Best: f(%.5f) = %.5f' % (sample[best_ix], sample_eval[best_ix]))
```

rand(100) generates a vector of 100 uniformly distributed in [0,1] random numbers.

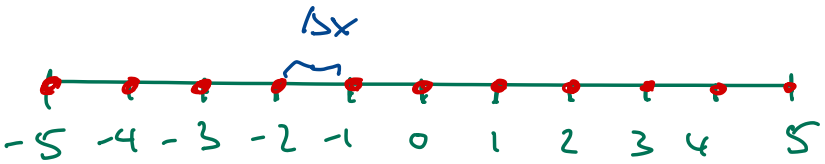
```
# get ready to plot
# sample input range uniformly at 0.1 increments
inputs = arange(r_min, r_max, 0.1)
# compute targets
results = f(inputs)
# create a line plot of input vs result
pyplot.plot(inputs, results)
# plot the sample
pyplot.scatter(sample, sample_eval)
# draw a vertical line at the best input
pyplot.axvline(x=sample[best_ix], ls='--', color='red')
pyplot.show()
```



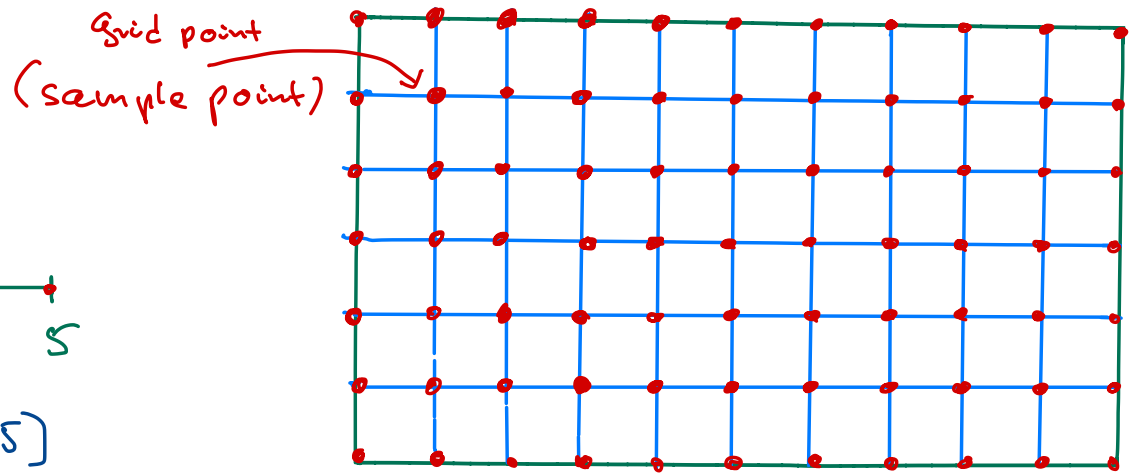
b) Grid Search (There is no randomness in this algorithm)

Instead of choosing the sample points $x_1, \ldots, x_N$ by random, they are distributed over the the feasible domain Ω in a regular way:

1. Place a uniform grid $x_1, \ldots, x_N$ over Ω.
2. Evaluate the objective function at these grid points.
3. The smallest/largest of the values of $f(x_1), \ldots, f(x_N)$ will be the max/min.



A uniform grid on [-5,5]
(in 1 dimension)



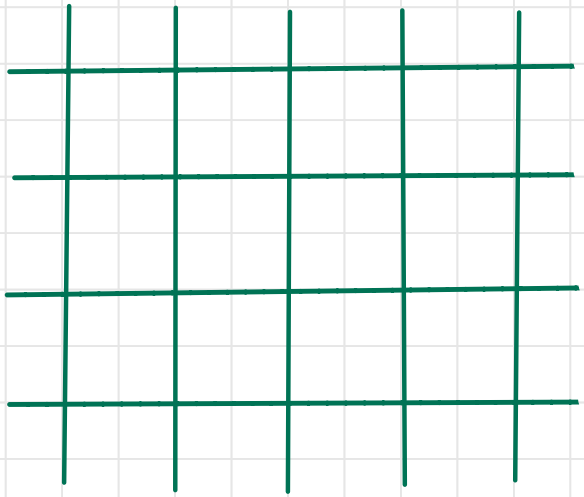
a uniform grid (red points)
in two dimensions.

Advantages:

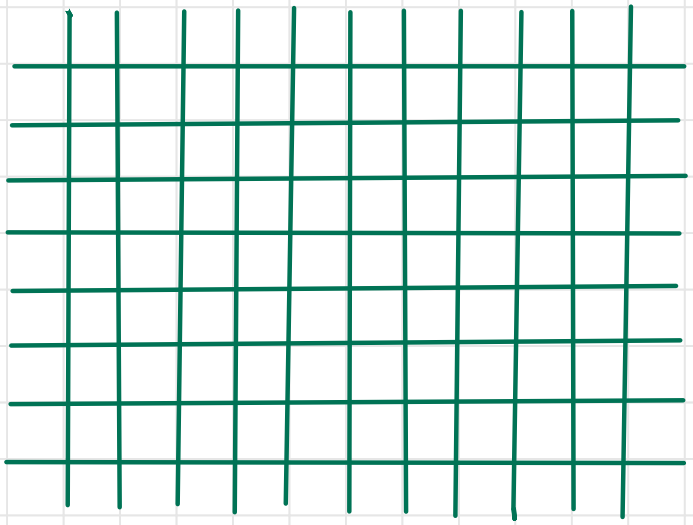
- This is a very simple algorithm. Easy to implement.
- Does not require much memory.
- Computations can be done in parallel.
- One can use grid search iteratvely. In a first step, identify regions in Ω which are 'good'. Then place a finer grid over the good regions and repeat the search.

Disadvantages:

- If the grid is too coarse, one may miss optimal solutions.
- The number of grid points grows exponentially with the dimension n. Thus, in high dimensions, a fine grid results in a large number of grid points, which can make the algorithm slow.



Coarse grid



fine grid.

```python
# example of grid search for function minimization
from numpy import arange
from numpy import meshgrid
from numpy.random import rand
from matplotlib import pyplot

# objective function
def f(x, y):
    return x**2.0 + y**2.0
```

$f(x,y) = x^2 + y^2$

function to be minimized

```python
# define  the feasible region Omega   (a square)
r_min, r_max = -5.0, 5.0
# generate a grid sample from the domain
sample = list()
step = 0.5
for x in arange(r_min, r_max+step, step):
    for y in arange(r_min, r_max+step, step):
        sample.append([x,y])
# evaluate the sample
sample_eval = [f(x,y) for x,y in sample]
# locate the best solution
best_ix = 0
for i in range(len(sample)):
    if sample_eval[i] < sample_eval[best_ix]:
        best_ix = i
# summarize best solution
print( 'Best: f(%.5f,%.5f) = %.5f' % (sample[best_ix][0],
                        sample[best_ix][1], sample_eval[best_ix]) )
```

$\Omega = [-5,5] \times [-5,5]$

→ distance between grid points

Create the list of grid points

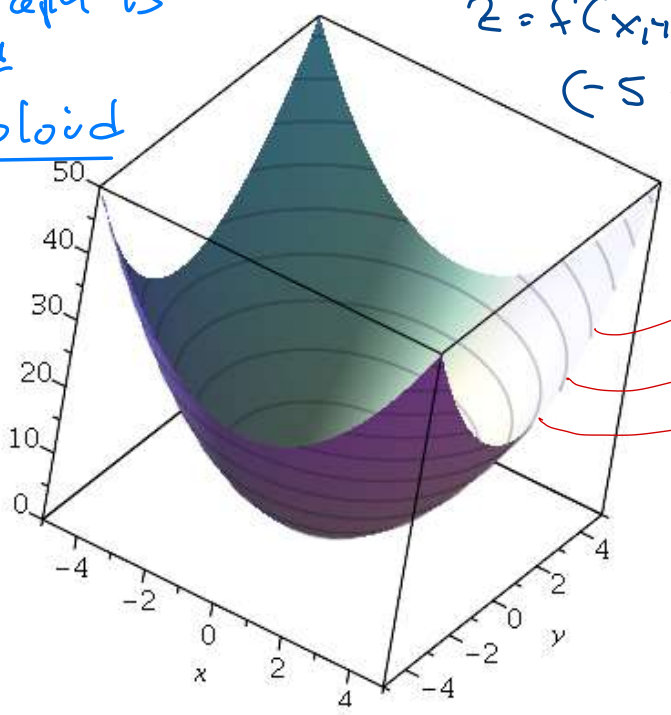
f(x,y) at all grid points (a list)

```
#
############  ready to plot
#
# create a mesh uniformly at 0.1 increments
xaxis = arange(r_min, r_max, 0.1)
yaxis = arange(r_min, r_max, 0.1)
x, y = meshgrid(xaxis, yaxis)
# compute targets
results = f(x, y)
# create a filled contour plot
my_contourplot = pyplot.contourf(x, y, results, levels=50, cmap='jet')
pyplot.colorbar(my_contourplot, label='Z values')
# plot the gridpoints as black circles
pyplot.plot([x for x,_ in sample], [y for _,y in sample], '.', color='black')
# draw the best result as a white star
pyplot.plot(sample[best_ix][0], sample[best_ix][1], '*', color='white')
# show the plot
pyplot.show()
```

this graph is called paraboloid

$z = f(x,y) = x^2 + y^2$

$(-5 \le x, y \le 5)$



3d-plot with contourlines

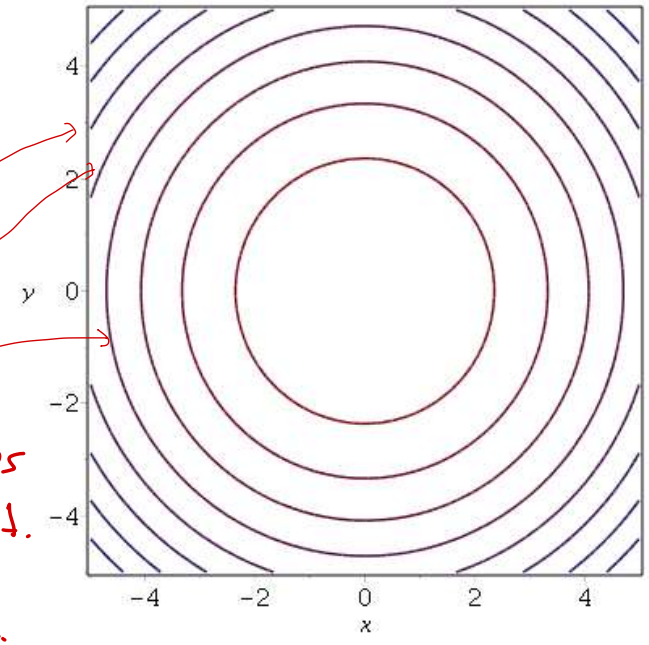
Contour lines

$f(x,y) = \text{Const.}$

Here:

$x^2 + y^2 = \text{Const.}$

(= Circles)

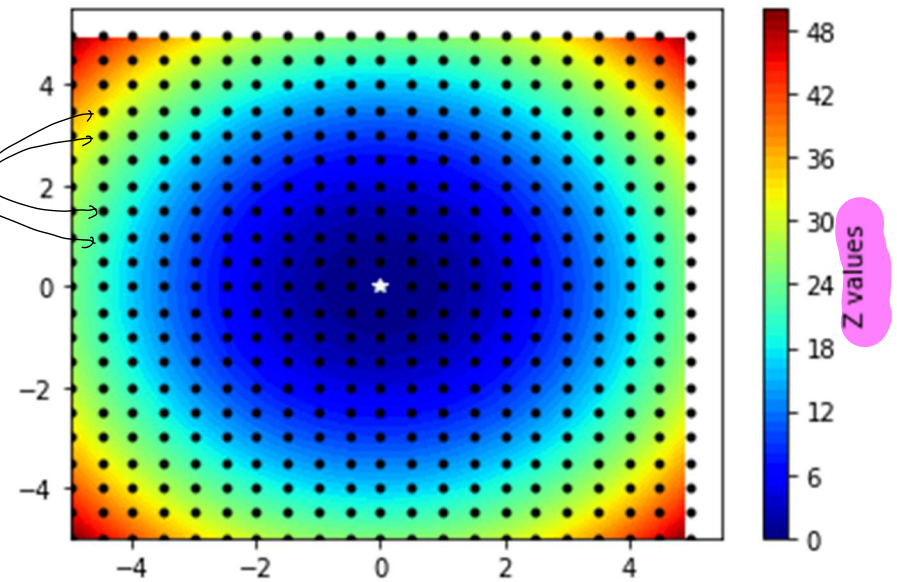


Contourplot

$$f(x) = x^2 + y^2$$

grid search: evaluate f at the grid points (x_k, y_k).

grid points (x_k, y_k)



contourplot with colormap

the colors indicate the values of $z = f(x,y)$

c) Neighborhood (local) naïve random search algorithm:

(Local neighborhood search)

one after another

In this algorithm, the points $x_1, \ldots, x_N$ are chosen *sequentially:*

Given any sample point x_k, the next sample point x_{k+1} is selected randomly from within a neighborhood $N(x_k)$ of x_k.

Neighborhood naïve random search for a minimization problem:

1. Set $k = 0$ and pick an initial x_0 in Ω.
2. Pick a candidate z_k close to x_k, at random.
   (z_k is from a neighborhood $N(x_k)$ of x_k.)
3. If $f(z_k) < f(x_k)$, then set $x_{k+1} = z_k$, else set $x_{k+1} = x_k$.
4. If the stopping criterion is satisfied, stop. $f(x_{k+1})$ is the optimal value.
5. Set $k := k + 1$ and go to step 2.

Example: Find the min. value of $\boxed{f(x) = x^2}$ on $\Omega = [-5, 5]$.

Sol: We choose a neighborhood

$$N(x_k) = [x_k - 1, x_k + 1]$$

① Choose $x_0 \in [-5, 5]$ by random

i.e. choose $x_0 = 3.40$

② choose $z_0 \in [x_0 - 1, x_0 + 1] = [2.40, 4.40]$ by random

$k = 0$: we choose $z_0 = 3.20$

Compare:

$$f(z_0) = 3.20^2 = 10.24$$

$$f(x_0) = 3.40^2 = 11.56$$

$\boxed{f(z_0) < f(x_0) \text{ !!}}$ Set $\boxed{x_1 = z_0 = 3.20}$

③ Choose $z_1 \in [x_1 - 1, x_1 + 1] = [2.20, 4.20]$ by random

$k = 1$:

We choose $z_1 = 4.09$

Compare: $f(z_1) = 4.09^2 = 16.7281$

$f(x_1) = 3.20^2 = 10.24$

$\boxed{f(z_1) > f(x_1) \text{ !!}}$ Keep $x_1 = 3.20$

So set $\boxed{x_2 = x_1 = 3.20}$

④ Choose $z_2 \in [x_2 - 1, x_2 + 1] = [2.20, 4.20]$ by random

$k = 2$

We choose $z_2 = 2.73$

Compare: $f(z_2) = 2.73^2 = 7.4529$

$f(x_2) = 3.20^2 = 10.24$

$\boxed{f(z_2) < f(x_2)\ !!}$ Set $\boxed{x_3 = z_2 = 2.73}$

$\vdots \qquad \vdots$

Continue this way

Advantages: (when compared to naïve random search)

- We need search across the whole search space, but come closer to a point of minimum with each iteration
- If the objective function f is "nice", fewer sample points x_k are required.

Disadvantage:

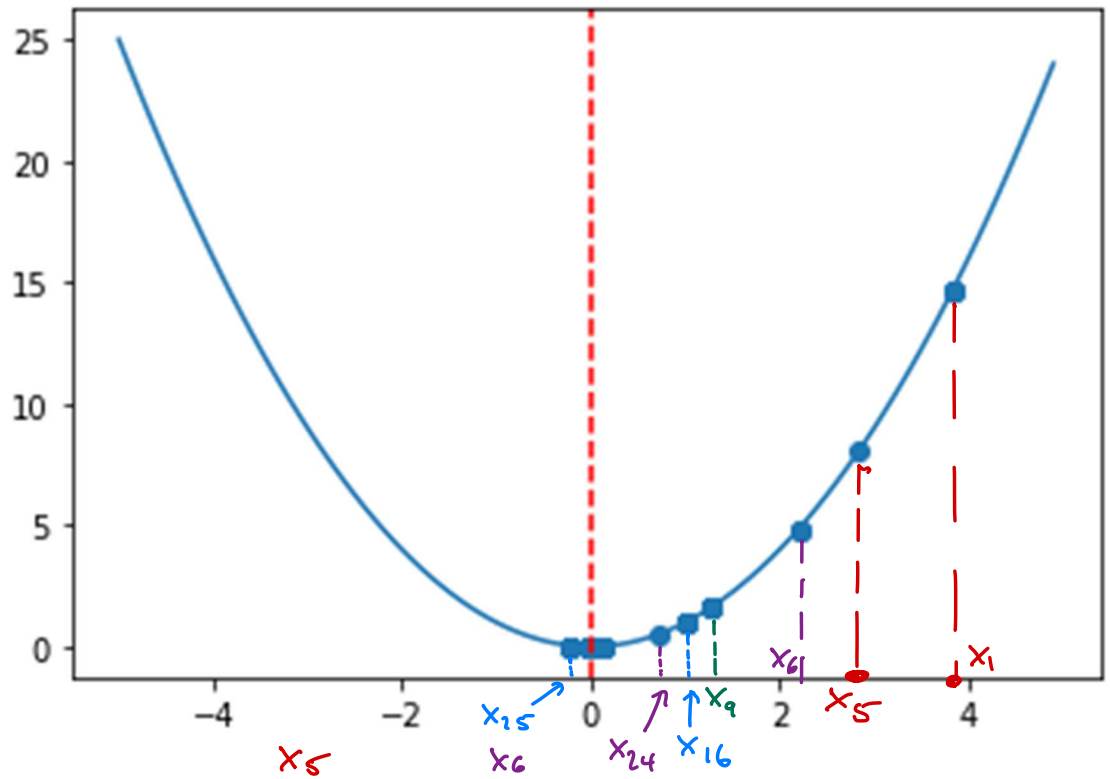
- We may end up at a **local** minimum, not an **absolute** minimum
- Computations can NOT be done in parallel.

```
# example of neighborhood naive random search
import numpy as np
from numpy import arange
from numpy.random import rand
from matplotlib import pyplot

# objective function
def f(x):
    return x**2.0

# the feasible domain Omega is an interval [r_min,r_max]
r_min, r_max = -5.0, 5.0
# size of neighborhood:  [x_k-eps,x_k+eps]
eps = 1.0;
#  stopping crterion: number of iterations
n_iter = 100
#   record all the sample points for plotting
sample_points=np.zeros( n_iter )
# initial sample point x_0
x_k  =  r_min + rand() * (r_max - r_min)
# Loop to construct sequence of sample points
for k in range( n_iter ):
    # case of uniform distribution
    z_k = x_k - eps + rand() * 2*eps
    # case of standard normal distribution
    # z_k = x_k + np.random.normal()
    #  test whether z_k is outside the feasable domain:
    if  (z_k < r_min):
```

```
            z_k = r_min
    if  (z_k > r_max ):
            z_k = r_max;
    if f(z_k) < f(x_k):
          x_k = z_k
#   record all the sample points for plotting
    sample_points[ k ] = x_k
#
# summarize best solution
print('Best: f(%.5f) = %.5f' % ( x_k, f(x_k) ) )
#
# get ready to plot
# sample input range uniformly at 0.1 increments
inputs = arange(r_min, r_max, 0.1)
# compute targets
results = f(inputs)
# create a line plot of input vs result
pyplot.plot(inputs, results)
# plot the sample points/values that we have moved through
sample_eval = f( sample_points  )
pyplot.scatter(sample_points, sample_eval)
# draw a vertical line at the best input
pyplot.axvline(x=x_k, ls='--', color='red')
pyplot.show()
#  print the sequence of sample points to show convergence towards the
location of the min
print(np.array_str( sample_points , precision=5, suppress_small=True))
```

Starting point: $\boxed{\texttt{x_0=3.83249}}$

Best: f(-0.01062) = 0.00011



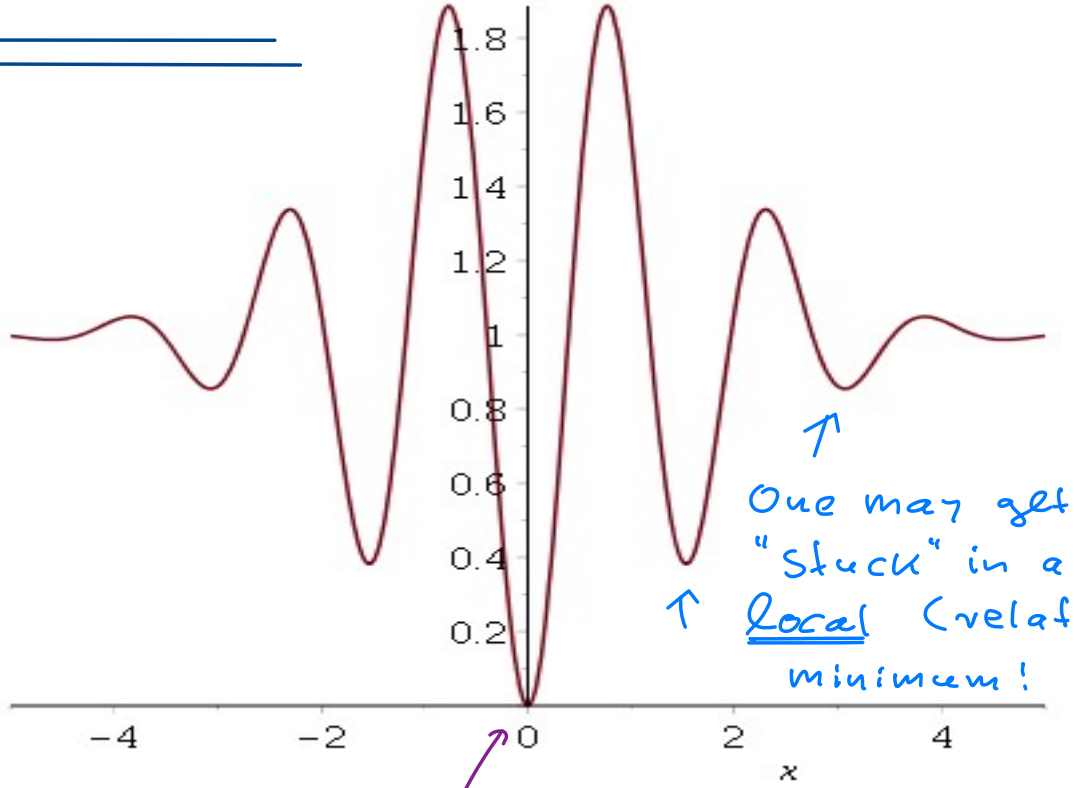
$x_1 = x_0$ $\quad$ $x_2 = x_0$

70 iterations

	x_5	x_6						
[3.83249	3.83249	3.83249	3.83249	2.84005	2.2031	2.2031	2.2031	
x_9 1.27975	1.27975	1.27975	1.27975	1.27975	1.27975	1.27975	1.02065	x_{16}
1.02065	1.02065	1.02065	1.02065	1.02065	1.02065	1.02065	0.72901	x_{24}
x_{25} -0.21257	-0.21257	-0.21257	-0.21257	-0.21257	-0.21257	-0.21257	0.13497	
0.13497	0.13497	0.13497	0.13497	-0.01914	-0.01914	-0.01914	-0.01914	
-0.01914	-0.01914	-0.01914	-0.01914	-0.01914	-0.01914	-0.01914	-0.01914	
-0.01914	-0.01914	-0.01914	-0.01914	-0.01914	-0.01914	-0.01914	-0.01914	
-0.01914	-0.01914	-0.01914	-0.01914	-0.01914	-0.01062	-0.01062	-0.01062	
-0.01062	-0.01062	-0.01062	-0.01062	-0.01062	-0.01062]			

Problems with this method

$$f(x) = 1 - e^{-0.2x^2}\cos(4x)$$



One may get "stuck" in a local (relative) minimum!

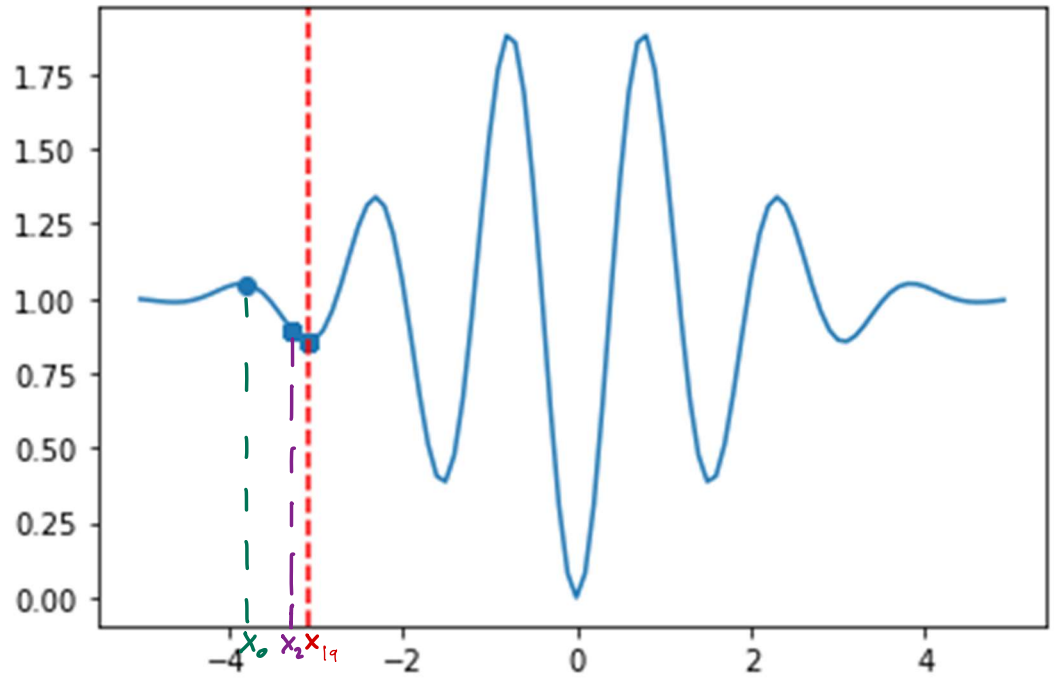
We want to find this absolute minimum

This function has many local max/min

The following example runs use a neighborhood of the form $[x_k - 1, x_k + 1]$ in which to search for z_k.

```
Starting point: x_0=-3.78477
Best: f(-3.05873) = 0.85443
```

end at a local min.



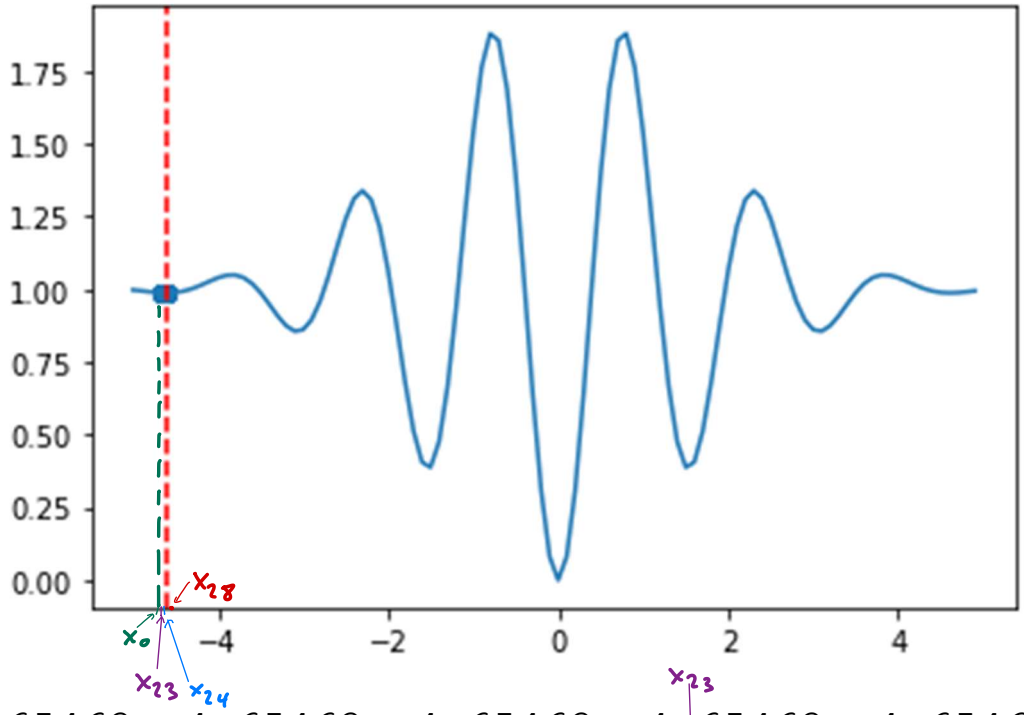

```
80 iterations (n_iter=80):
```

x_1 x_2 x_{19}

```
[-3.78477 -3.25686 -3.25686 -3.25686 -3.25686 -3.25686 -3.25686 -3.25686
 -3.25686 -3.25686 -3.25686 -3.25686 -3.25686 -3.25686 -3.25686 -3.25686
 -3.25686 -3.25686 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873
 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873
 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873
 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873
 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873
 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873
 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873
 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 -3.05873 ]
```

```
Starting point: x_0=-4.65463
Best: f(-4.60498) = 0.98692
```

end up at a local min.

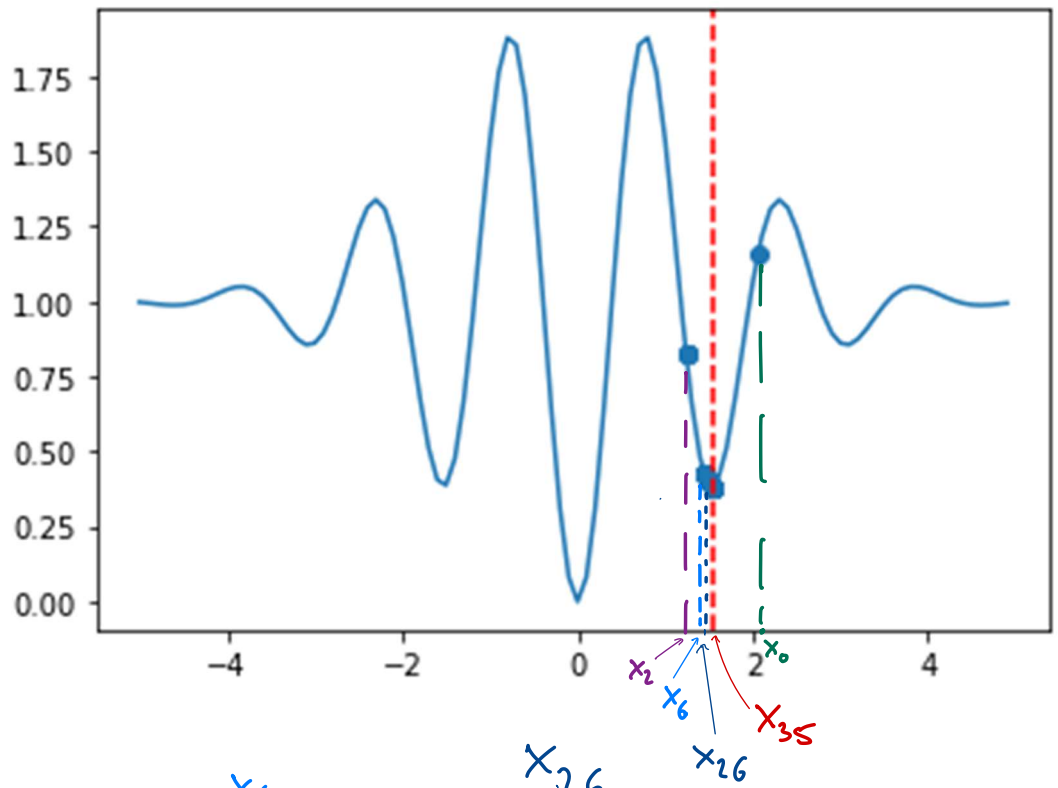


80 iterations (n_iter=80):

```
[-4.65463 -4.65463 -4.65463 -4.65463 -4.65463 -4.65463 -4.65463 -4.65463
 -4.65463 -4.65463 -4.65463 -4.65463 -4.65463 -4.65463 -4.65463 -4.65463
 -4.65463 -4.65463 -4.65463 -4.65463 -4.65463 -4.65463 -4.62414 -4.61860 ← x_24
 -4.61860 -4.61860 -4.61860 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498
 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498
 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498
 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498
 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498
 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 -4.60498 ]
```

```
Starting point: x_0=2.05952
Best: f(1.53602) = 0.38219
```

End up at a local min.




```
80 iterations (n_iter=80):
```

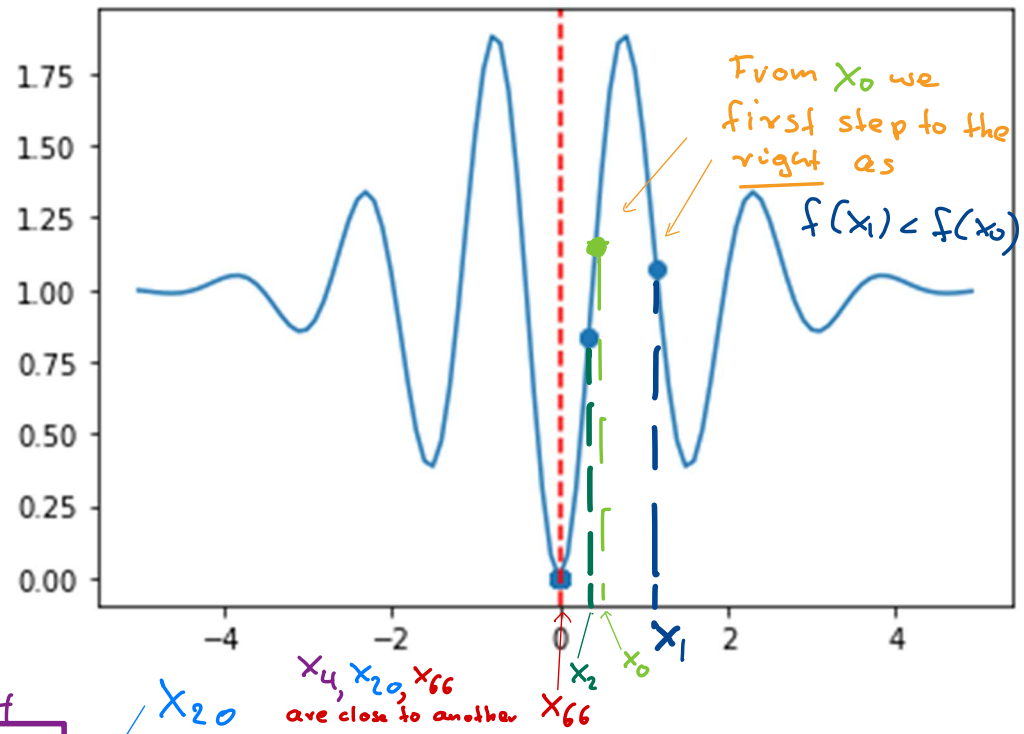
x_2 x_6 x_{26} x_{36}

```
[2.05952 1.23798 1.23798 1.23798 1.23798 1.44574 1.44574 1.44574 1.44574
 1.44574 1.44574 1.44574 1.44574 1.44574 1.44574 1.44574 1.44574 1.44574
 1.44574 1.44574 1.44574 1.44574 1.44574 1.44574 1.44574 1.46421 1.46421
 1.46421 1.46421 1.46421 1.46421 1.46421 1.46421 1.46421 1.53602 1.53602
 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602
 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602
 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602
 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602
 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 1.53602 ]
```

```
Starting point: 0=0.47310
Best: f(0.00152) = 0.00002
```

end up at the <u>absolute</u> min.

the starting point x_0 is close enough to the point of min. [=0]




```
80 iterations (n_iter=80):
```

x_1 x_2 x_4 x_{20}

```
[ 1.15379  0.35128  0.35128 -0.0094  -0.0094  -0.0094  -0.0094  -0.0094
 -0.0094  -0.0094  -0.0094  -0.0094  -0.0094  -0.0094  -0.0094  -0.0094
 -0.0094  -0.0094  -0.0094  -0.00685 -0.00685 -0.00685 -0.00685 -0.00685
 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685
 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685
 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685
 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685
 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685 -0.00685
 -0.00685  0.00152  0.00152  0.00152  0.00152  0.00152  0.00152  0.00152
  0.00152  0.00152  0.00152  0.00152  0.00152  0.00152  0.00152  0.00152 ]
```

x_{66}

Question: How can we avoid ending up at a **local** minimum/maximum ?

Some possible solutions are:

i.e. Choose $z_k \in [x_k - \varepsilon, x_k + \varepsilon]$ where ε is large.

1. Allow the next sample point z_k to be far from x_k (i.e. make the neighbourhood $N(x_k)$ large).

   (This may require a larger number of iterations to be performed, because one may jump across the whole feasible domain= search space)

2. Allow the next sample point z_k to be far from x_k , but with only low probability. (i.e. choose z_k Gaussian pdf.

3. **Repeated Search.** Repeat the search algorithm several times, each time with a different starting point.

4. Allow that x_{k+1} be a "worse" point than x_k, with some restrictions. (in a minimization problem, "worse" means $f(x_{k+1}) > f(x_k)$)

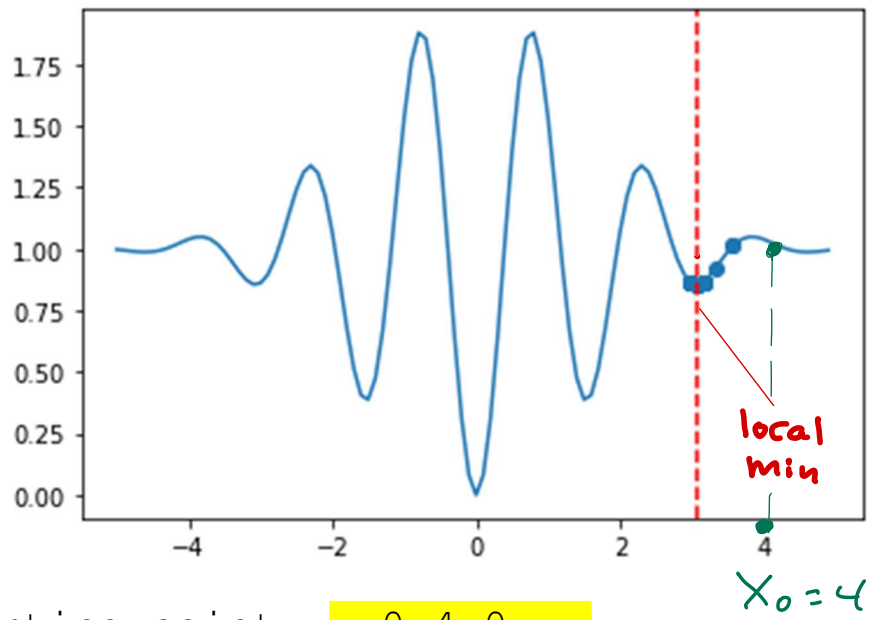
   (e.g. the *Simulated Annealing* algorithm. (which we will study)

Let's look at these possibilities by examples.

1. Make the neighbourhood $N(x_k)$ large.

$\varepsilon = 1$

i) Search neighborhood $z_k \in [x_k - 1, x_k + 1]$, uniform distribution

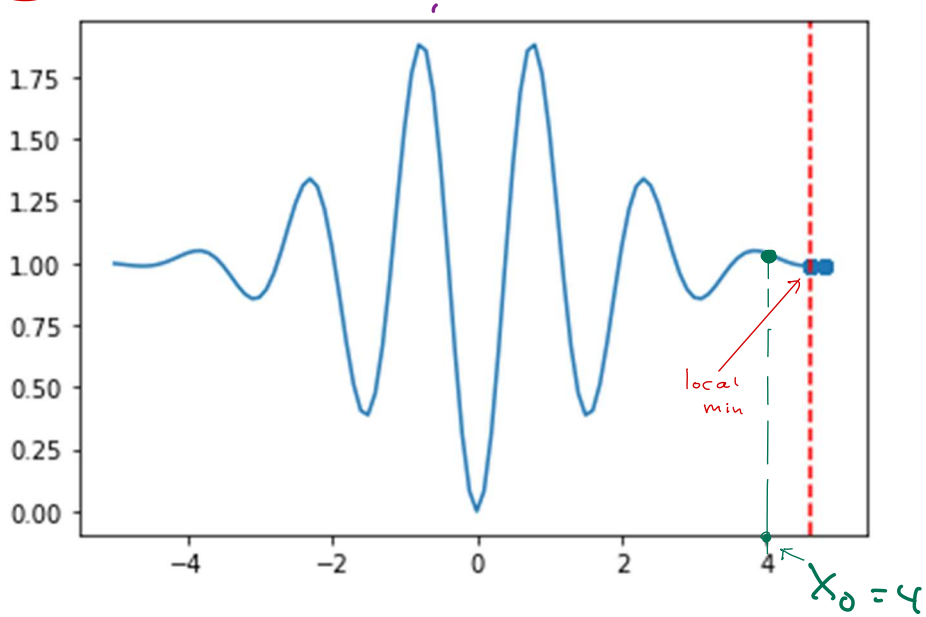



```
Starting point: x_0=4.0
        x_1 = 3.57106
        x_3 = 3.32028
        x_4 = 2.96592
        x_19= 3.17664
        x_33= 3.16511
        x_57= 3.07443
Solution: f(3.07443) = 0.85441
```

End up at a local min

Typical possible solution

among 50 runs of the algorithm

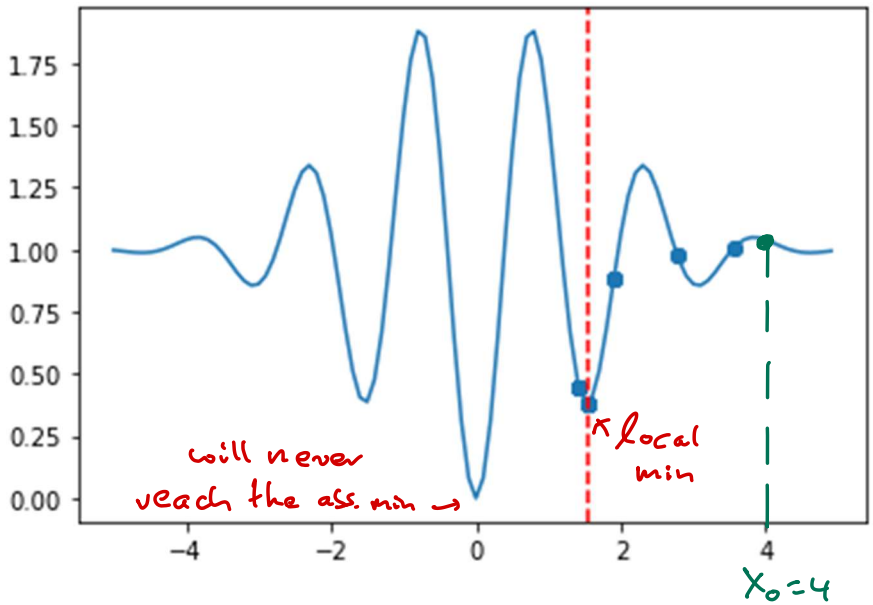



```
Starting point: x_0=4.00000
        x_1 = 4.80698
        x_6 = 4.59634
Solution: f(4.59634) = 0.98693
```

after 6 iterations, end up at a local min point that is far away from the assolute min.

The worst possible solution

among 50 runs of the algorithm



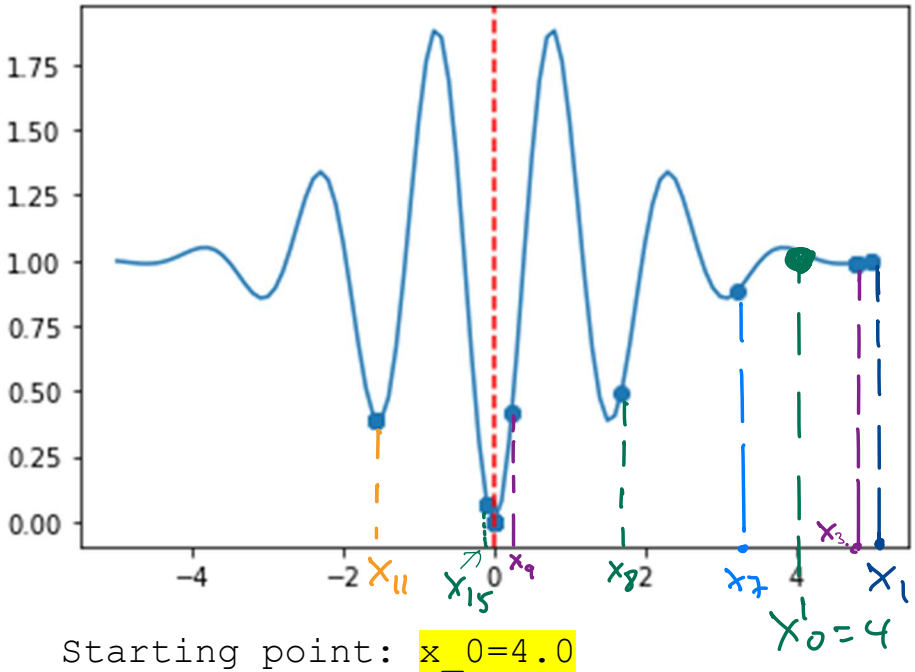

```
Starting point: x_0=4.0
    x_1 = 3.54576
    x_3 = 2.77038
    x_6 = 1.9046
    x_11= 1.42397
    x_21= 1.54584
    x_50= 1.53659
Solution: f(1.53659) = 0.38221
```

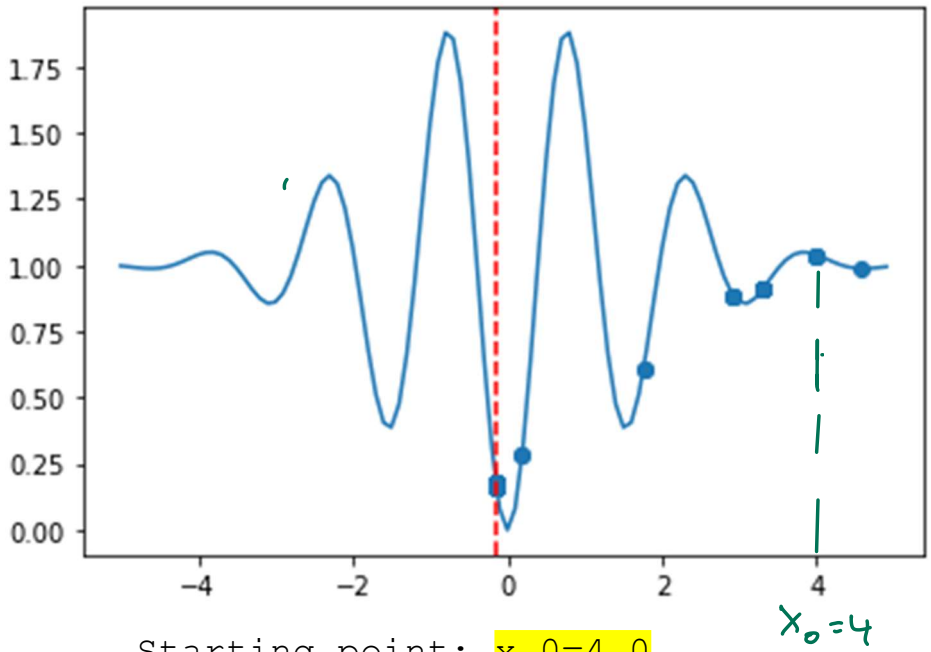
The best possible solution is only a local min

(one can never reach the absolute min. point at x=0)

choose $\varepsilon = 2$

ii) Enlarge the search neighborhood to $z_k \in [x_k - 2, x_k + 2]$, uniform distribution






```
Starting point: x_0=4.0
        x_1 = 5.0
        x_3 = 4.79653
        x_7 = 3.23482
        x_8 = 1.68551
        x_9 = 0.23388
        x_11= -1.5604
        x_15= -0.09326
        x_20= -0.01138
        x_40= -0.00738
Solution:: f(-0.00738) = 0.00045
```

overshoot

Move over large region in search space

When ε is large, one moves all over Ω.

```
Starting point: x_0=4.0
        x_1 = 4.0
        x_5 = 4.55422
        x_6 = 3.29548
        x_14= 2.91816
        x_17= 1.7586
        x_19= 0.19188
        x_21= -0.14938
        x_75= -0.14302

Solution: f(-0.14302) = 0.16266
```

Solution of low precision

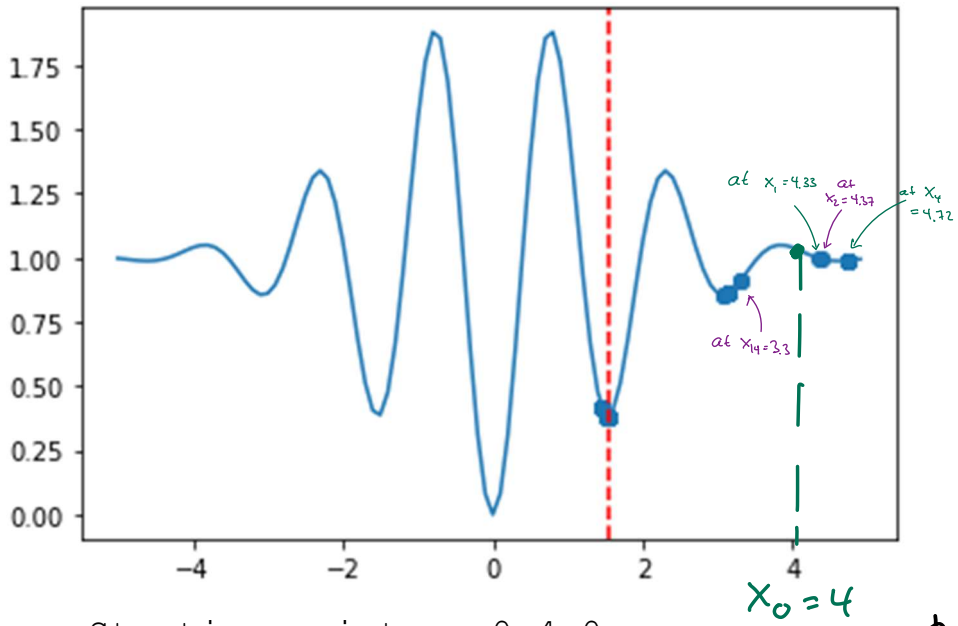
Moved to local optimization as needed for good precision.

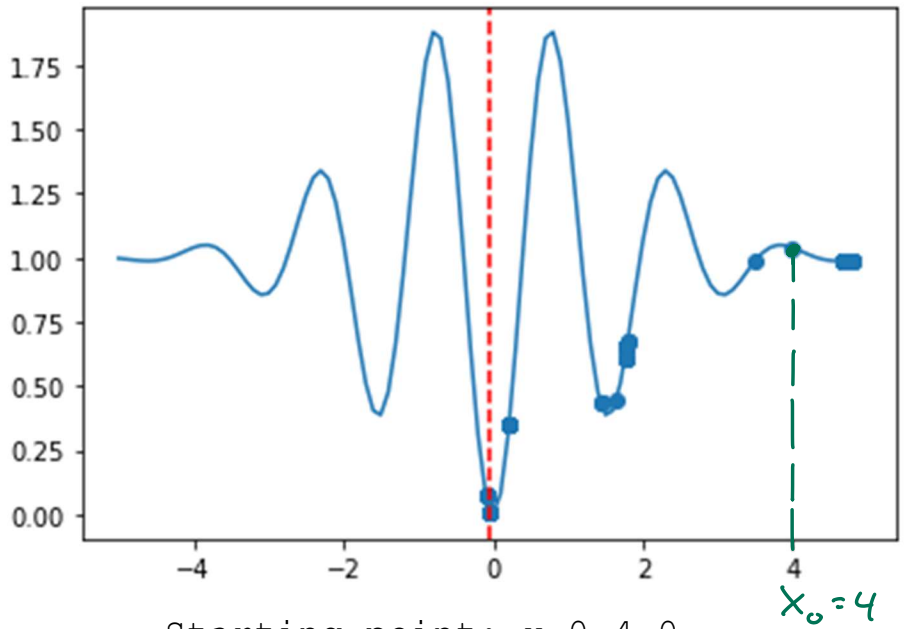
with $\varepsilon = 2$

In 50 runs, the absolute min was always reached.

However, sometimes with low precision.

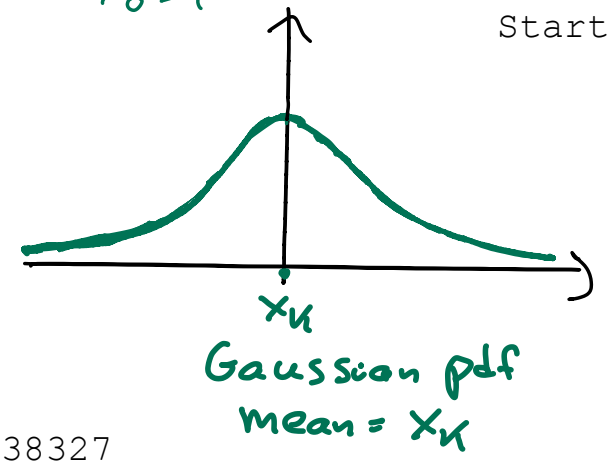
2. Allow z_k to be far from x_k, but with <u>only low probability</u>.
(Choose z_k with Gaussian distribution, mean=x_k, SD=0.8)






```
Starting point: x_0=4.0
        x_1 = 4.32696
        x_2 = 4.37061
        x_4 = 4.71978
        x_14= 3.30022
        x_17= 3.13442
        x_42= 3.07127
        x_45= 1.44985
        x_53= 1.5156
        x_58= 1.54762
Solution: f(1.54762) = 0.38327
```

Local min only




```
Starting point: x_0=4.0
        x_1 = 4.0
        x_2 = 4.78117
        x_7 = 4.7127
        x_16= 4.69391
        x_28= 4.67959
        x_38= 3.4943
        x_39= 1.79788
        x_42= 1.78633
        x_55= 1.75859
        x_61= 1.64667

        x_62= 1.4327
```

```
x_65= 0.21555
x_78= -0.09363
x_85= -0.0369
Solution: f(-0.03690) = 0.01114
```

With a choice of SD=0.8, the absolute min is reached in most of 100 runs
With a choice of SD=1.0, the absolute min is reached in all runs

Possible modification: decrease the SD with each iteration

3. **Repeated Search.**

Assignment: Modify the code of

```
# example of neighborhood naive random search
```

by looping over starting points -4.5, -3.5, -2.5, ….. 2.5, 3.5, 4.5

Then select the best of the solutions.

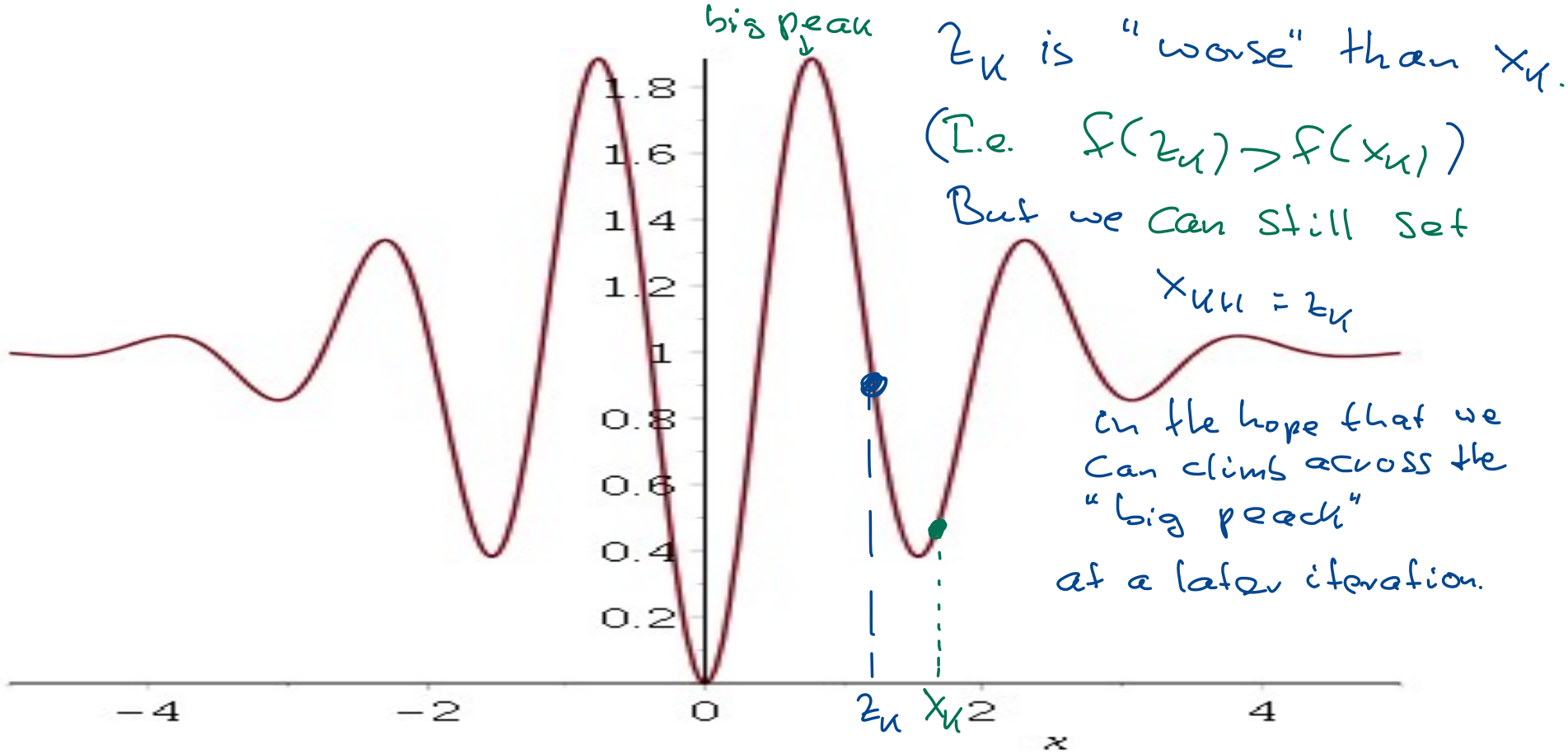
4. **Simulated Annealing**

We modify the `neighborhood naive random search:`

As before, when $f(z_k) \le f(x_k)$ then we set $x_{k+1} = z_k$

However, if $f(z_k) > f(x_k)$ then we toss a coin whose probability of (HEAD) equals $p = p(k, x_k, z_k)$. ← p depends on iteration k, z_k and x_k.

If the toss results in HEAD, then we set $x_{k+1} = z_k$, (so that $f(x_{k+1}) > f(x_k)$) otherwise we set $x_{k+1} = x_k$.



Note: i) Because $f(x_{k+1}) > f(x_k)$ is possible, we need to keep track of the *best* x_k so far achieved in the algorithm; call it x_{best}.

ii) The probability $p = p(k, x_k, z_k)$ of obtaing a HEAD will depend on

- the iteration k , the value of $f(x_k)$ and the value $f(z_k)$.

A common choice is

$$p = p(k, x_k, z_k) = \min\left(1,\ e^{-[\,f(z_k) - f(x_k)\,]/T_k}\right) \qquad 0 < p \leq 1$$

where

$$T_k = \frac{T_0}{k+1} \qquad \text{("temperature")}$$

and T_0 is a problem-dependent constant.

<u>We see:</u> During the first iterations (k small), T_k is relatively large, so that $e^{-[f(z_k) - f(x_k)]/T_k}$ is far from zero: p is far from zero, and the probability of a HEAD is large.

As k increases, the probability p of a HEAD decreases towards zero. with each iteration.

Also, when $f(z_k)$ is much larger than $f(x_k)$, then $-\left[f(z_k)-f(x_k)\right]/T_k$ is large negative, so that the probability p of a HEAD is small.

Other choices for T_k are:

$$T_k = \frac{T_0}{\ln(k+2)}, \qquad T_k = \alpha^k T_0, \quad 0 < \alpha < 1$$

How to toss a coin with probability of HEAD = p.

① Pick a uniformly distributed number r in $[0, 1]$

② If $r < p \Rightarrow$ HEAD

If $r > p \Rightarrow$ TAIL

Simulated Annealing Algorithm (for minimization)

1. Set $k = 0$ and pick an initial x_0 in Ω, $x_{\text{best}} = x_0$.
2. Pick a candidate z_k close to x_k, at random.
   (z_k is from a neighborhood $N(x_k)$ of x_k.)
3. If $f(z_k) < f(x_k)$ then set $x_{k+1} = z_k$, $x_{\text{best}} = z_k$,
   else { compute the probability $p = p(k, x_k, z_k)$
   coin toss: pick a number r, $0 \le r \le 1$, (unif. distr.)
   If $r < p$ ("HEAD") then $x_{k+1} = z_k$,
   else ("TAIL") $x_{k+1} = x_k$ }

   Allow $f(x_{k+1}) > f(x_k)$ with Probability p.
4. If the stopping criterion is satisfied, stop. $f(x_{\text{best}})$ is the optimal value.
5. Set $k := k + 1$ and go to step 2.

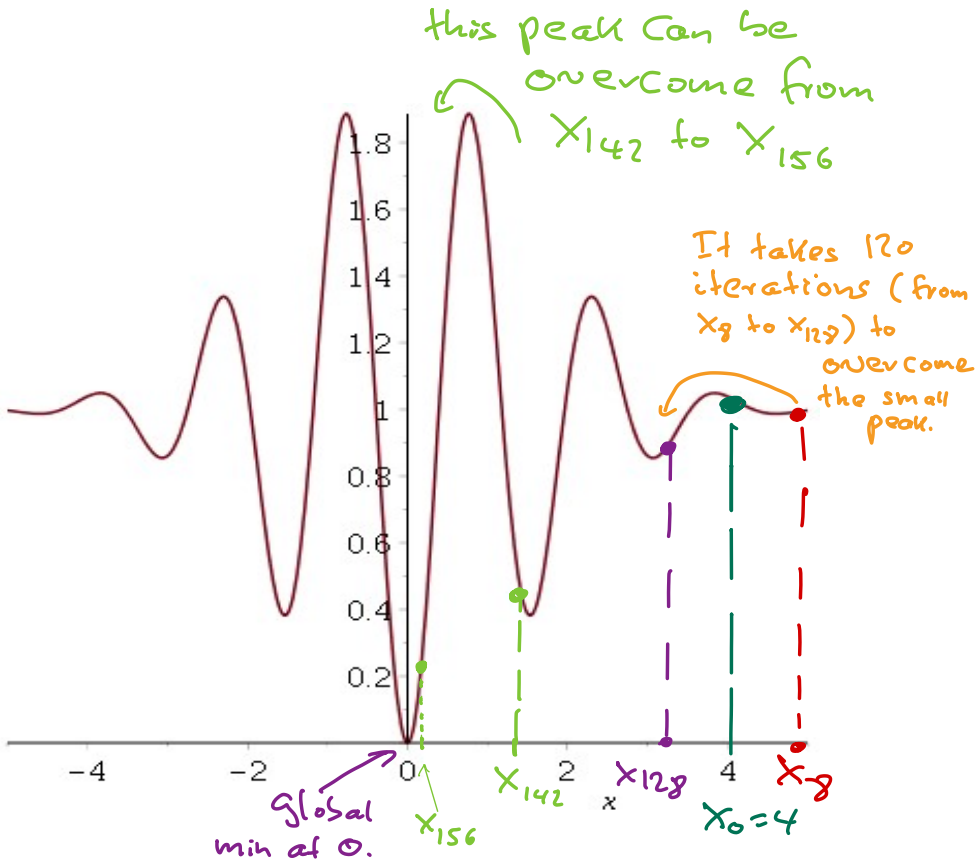
1st Column: iteration K

Step	f(x_best)
x_0 = 4.0	
>1	f([4.00854361]) = 1.03809
>4	f([4.31634496]) = 1.00032
>5	f([4.46008078]) = 0.99003
>6	f([4.46080896]) = 0.99000
>7	f([4.64264068]) = 0.98710
>8	f([4.6174176]) = 0.98694
>128	f([3.23913579]) = 0.88657
>129	f([3.09792609]) = 0.85554
>138	f([1.88303743]) = 0.84436
>141	f([1.72355168]) = 0.54784
>142	f([1.566317]) = 0.38788
>156	f([0.17438372]) = 0.23822
>158	f([-0.0859482]) = 0.05991
>172	f([0.01610672]) = 0.00213
>237	f([0.01472293]) = 0.00178
>255	f([0.00664156]) = 0.00036
>277	f([-0.00033045]) = 0.00000
>991	f([3.77760799e-05]) = 0.00000

x_k $f(x_k)$

Solution:

f([3.77760799e-05]) = 0.000000



Reach the global min after many iterations. (K=277)